

تمرین‌های کلاسی هفته سوم - 02

بخش 1 – موتور تشخیص سلیقه موسیقی

۱. یک اپ پخش موسیقی، هر آهنگ را با سه ویژگی نشان می‌دهد (تمپو نرمال‌شده، انرژی، قابلیت حرکات موزون):

```
song = np.array([0.7, 0.9, 0.6])
user_taste = np.array([0.6, 0.8, 0.5])
```

اول ضرب elementwise این دو بردار را حساب کنید، بعد dot product را. کدام یک عددی تکی برمی‌گرداند که می‌تواند «میزان تطابق» آهنگ با سلیقه کاربر باشد؟

۲. حالا یک بایاس هم اضافه می‌کنیم – چون کاربر الان حال و حوصله بیشتری برای موسیقی دارد:

```
w = np.array([0.6, 0.8, 0.5])
b = 0.3
```

با فرمول نوروں خطی، امتیاز نهایی «پیشنهاد بده یا نده» را برای همان آهنگ بالا در یک خط حساب کنید.

۳. ● سه آهنگ داریم:

```
song1 = np.array([0.9, 0.1])
song2 = np.array([0.85, 0.15])
song3 = np.array([-0.9, -0.1])
```

با cosine similarity مشخص کنید کدام دو آهنگ از نظر «سبک» به هم نزدیک‌ترند و کدام تقریباً ضد هم هستند. چرا اینجا فقط dot product خام کافی نیست؟

۴. یک پلی‌لیست جدید با 5 آهنگ به اپ اضافه شده

```
(songs, shape (5,3))
```

و می‌خواهید امتیاز تطابق همه آن‌ها با سلیقه همان کاربر را همزمان حساب کنید. با یک خط کد (بدون for) این کار را انجام دهید.

بخش ۲ – امتیازدهی شخصیت‌های بازی

۵. یک بازی موبایلی ۴ شخصیت دارد. برای هر شخصیت، ۳ ویژگی اولیه ذخیره شده است:

- Power
- Speed
- Defense

این اطلاعات در ماتریس زیر قرار دارند:

```
stats = np.random.rand(4, 3)
```

در این ماتریس:

- هر سطر نشان‌دهنده یک شخصیت است.
- هر ستون یکی از ویژگی‌های Power، Speed و Defense را نشان می‌دهد.

حالا بازی می‌خواهد با استفاده از این سه ویژگی، برای هر شخصیت ۵ امتیاز جدید (مانند Attack Score، Magic Score، Survival Score و ...) محاسبه کند. برای این کار از ماتریس زیر استفاده می‌شود:

```
skill_matrix = np.random.rand(3, 5)
```

هر ستون از `skill_matrix` مشخص می‌کند که چگونه Power، Speed و Defense با هم ترکیب شوند تا یکی از این امتیازهای جدید به دست آید.

الف) قبل از اجرای کد، فقط با توجه به shape دو ماتریس، حدس بزنید shape حاصل عبارت زیر چه خواهد بود:

```
stats @ skill_matrix
```

ب) کد را اجرا کنید و حدس خود را بررسی کنید.

ج) توضیح دهید که سطرها و ستون‌های ماتریس خروجی چه مفهومی دارند.

۶. حالا یک مثال واقعی‌تر را در نظر بگیرید. فرض کنید اطلاعات ۴ شخصیت به صورت زیر ذخیره شده است:

```
X_chars = np.array([
    [80, 40, 60],
    [30, 90, 50],
    [70, 70, 70],
    [95, 20, 40]
])
```

هر سطر مربوط به یک شخصیت و هر ستون به ترتیب نشان‌دهنده Speed، Power، Defense است.

فرض کنید یک لایه با ۳ نورون قرار است برای هر شخصیت سه مقدار جدید تولید کند:

- Attack Score
- Defense Score
- Final Speed Score

وزن‌ها و bias این لایه به صورت زیر تعریف شده‌اند:

```
W_skills = np.random.randn(3, 3)
b_skills = np.array([1.0, -0.5, 0.2])
```

الف) خروجی این لایه را با استفاده از رابطه زیر محاسبه کنید:

```
Z = X_chars @ W_skills + b_skills
```

ب) shape ماتریس Z را بررسی کنید.

ج) توضیح دهید هر سطر و هر ستون از Z چه مفهومی دارد.

۷.  طراح بازی به اشتباه ماتریس وزن را با shape نادرست تعریف کرده است:

```
W_wrong = np.random.randn(5, 3)
```

به همین دلیل عبارت زیر اجرا نمی‌شود:

```
X_chars @ W_wrong
```

الف) بدون تغییر مقادیر **W_wrong** و فقط با استفاده از `transpose`، ضرب ماتریسی را امکان‌پذیر کنید.

ب) `shape` خروجی جدید را بنویسید.

ج) توضیح دهید که در این حالت، ۵ ستون خروجی چه مفهومی دارند و هر شخصیت اکنون چند خروجی تولید می‌کند.

بخش 3 – اپ کوئیز و تریویا

۸. یک اپ کوئیز، سؤالات و پاسخ‌های درست‌شان را در دو آرایه جدا نگه می‌دارد:

```
questions = np.array(['Q1', 'Q2', 'Q3', 'Q4', 'Q5'])
answers    = np.array(['B', 'D', 'A', 'C', 'B'])
```

می‌خواهید ترتیب نمایش سؤالات به هر کاربر تصادفی باشد، اما هر سؤال باید همیشه با پاسخ درست خودش جفت بماند. درستی جفت‌ماندن را چاپ کنید.

۹. یک دسته سؤال جدید هم طراحی شده و باید به بانک سؤالات قبلی اضافه شود:

```
new_questions = np.array(['Q6', 'Q7'])
new_answers   = np.array(['A', 'D'])
```

این دو سؤال جدید و پاسخ‌هایشان را به آرایه‌های اصلی اضافه کنید. حالا فرض کنید هر سؤال یک امتیاز سختی هم دارد

```
difficulty = np.array([1,2,1,3,2,2,3])
```

چطور می‌توانید سؤال، پاسخ و سختی را در قالب یک جدول 2 بعدی (7 سؤال $\times$ 3 ستون) کنار هم بچینید؟
(راهنمایی: `vstack` روی نسخه‌های transpose شده)

بخش 4 – داشبورد سنسورهای خانه هوشمند

۱۰. یک سیستم امنیتی هوشمند از 6 دوربین، هر کدام یک عکس 3×3 پیکسلی می‌گیرد این آرایه را `camera_frames` و با `rand` آن را بسازید.

برای ارسال داده‌ها به سرور، هر تصویر باید به یک بردار 9 عضوی تبدیل شود، در حالی که تعداد تصاویر تغییری نکند.

الف) آرایه را به بعد (6,9) تبدیل کنید.

ب) همین کار را یک بار دیگر انجام دهید، اما به جای نوشتن عدد 9 از 1- استفاده کنید و درباره نتیجه با هم گروهی های خود صحبت کنید.

۱۱. یک جدول لاگ داریم شامل 6 سنسور که 4 ساعت معیارهای مختلف را اندازه‌گیری کرده است. برای نمایش در داشبورد، باید محور زمان و محور سنسور را جابه‌جا کنید. `shape` جدید را بررسی کنید.

۱۲.  برای یک هفته داده داریم: 7 روز $\times$ 4 بازه زمانی $\times$ 3 سنسور دما:

```
week_temps = np.arange(84).reshape(?, ?, ?)
```

`shape` خود آرایه و `week_temps.T` `shape` را قبل از اجرا حدس بزنید. ترنسپوز روی آرایه سه‌بعدی دقیقاً با کدام محورها چه کاری کرد.

بخش 5 – دستیار صوتی

۱۳. یک دستیار صوتی، امتیاز اطمینان خودش را برای تشخیص 5 فرمان صوتی داده:

```
confidence = np.array([0.9, 0.4, 0.7, 0.95, 0.3])
```

برای اینکه این آرایه با خروجی مدل (که shape آن (5,1) است) مقایسه پذیر شود، آن را به یک بردار ستونی تبدیل کنید.

۱۴. خروجی مدل برای یک فرمان صوتی تکی به این شکل عجیب برگشته (چون کد برای batch نوشته شده بود):

```
single_command_output = np.array([[[0.1, 0.7, 0.2]]]) # shape: (1, 1, 3)
```

این را به بردار ساده (3,) تبدیل کنید. سپس کاری کنید که فقط یکی از دو بعد اضافه حذف شود.

بخش 6 – کنترل برق هوشمند

۱۶. یک کنترل برق هوشمند، به خاطر نویز سنسور، گاهی عدد منفی برای مصرف لحظه‌ای ثبت می‌کند که از نظر فیزیکی بی‌معناست:

```
power_readings = np.array([2.1, -0.5, 3.4, -1.1, 0, 5.6])
```

همه مقادیر منفی را صفر کنید و بقیه را دست‌نخورده نگه دارید.

۱۷. خروجی خام یک لایه کوچک که «سطح فعالیت» چند دستگاه خانگی را پیش‌بینی می‌کند ماتریس زیر است:

```
[[-0.4, 1.2, 0.0], [2.1, -1.5, 0.3]]
```

ReLU را روی این ماتریس پیاده کنید. در عمل با مقادیر منفی و صفر چه رفتاری دارد؟

بخش 7 – تشخیص ژانر فیلم

۱۸. یک مدل ساده، برای 3 فیلم، امتیاز خام (logits) روی 3 ژانر (اکشن، درام، کمدی) داده:

```
logits = np.array([
```

```
[2.5, 0.3, 0.1],  
[0.2, 0.1, 3.5],  
[1.0, 1.2, 0.9]  
)
```

این logits را به احتمال (softmax) تبدیل کنید طوری که مجموع هر سطر دقیقاً 1 شود. بعد یک بار `keepdims=True` را بردارید و ببینید کجای محاسبه خراب می‌شود.

۱۹. از روی همان ماتریس احتمالات:

- با `argmax(axis=1)` ژانر پیش‌بینی‌شده هر فیلم را پیدا کنید.
- با `max(axis=1)` میزان اطمینان مدل به آن پیش‌بینی را هم چاپ کنید.

۲۰. 🟡 همین را یک بار با `axis=0` امتحان کنید. توضیح بدهید این بار خروجی چه معنایی دارد و چرا برای «پیدا کردن ژانر هر فیلم» `axis` اشتباهی است.

بخش 8 – سبد خرید آنلاین 🟢

۲۱. یک تیم دیگر گزارش داده که کد زیر یک باگ عجیب دارد:

```
cart_prices = np.array([120, 45, 300, 15, 80])  
discount_items = cart_prices[:2]  
discount_items[0] = 0  
print(cart_prices)
```

خروجی `print` چه می‌شود و چرا؟ سپس کد را طوری بازنویسی کنید که تغییر `discount_items` هیچ اثری روی `cart_prices` اصلی نگذارد.

🎁 پروژه نهایی: طبقه‌بند حال‌وهوای آهنگ

یک اپ موسیقی 8 آهنگ جدید دارد که هر کدام با 4 ویژگی خام توصیف شده‌اند

```
(X_songs, shape (8,4))
```

و می‌خواهد هر کدام را به یکی از 3 دسته («شاد»، «آرام»، «غمگین») برچسب بزند:

1. آهنگ‌ها را shuffle کنید.
 2. از یک لایهٔ اول با 5 نورون رد کنید و ReLU بزنید.
 3. خروجی را به یک لایهٔ دوم با 3 نورون (متناظر با 3 حال‌وهوا) بدهید و softmax بزنید.
 4. با argmax، حال‌وهوای نهایی هر آهنگ را استخراج و چاپ کنید.
- (وزن‌ها و بایاس‌های هر دو لایه را خودتان با randn بسازید.)